

UNIT 5

Learn Smart – Part 2 of 2

CO Mapped: CO5

Research Based Problems Mapped: 2,3 & 4

Type: Practical + Analytical Activity (Project Part 2 of 2 — Builds on Activity 9)

Problem Statement

In Activity 10 you will complete the LearnSmart project by building the full recommendation layer that uses Activity 9 outputs to generate personalised recommendations through three approaches:

- Content-Based Filtering – recommend content similar to what the learner has viewed, filtered by difficulty level.
- Collaborative Filtering – recommend content that similar learners have rated highly (Matrix Factorization + Neighbourhood model).
- Sentiment-Adjusted Hybrid Recommender – combine both approaches, weighted by community sentiment from user reviews.

Problems solved in this activity:

- Learners receive no personalised content list — only a generic course catalogue.
- Content-based systems alone suffer from the 'filter bubble' problem (over-specialisation).
- Collaborative filtering alone suffers from the cold-start problem for new learners.
- No mechanism exists to factor in community feedback (positive/negative reviews) into rankings.

Objective

Complete the LearnSmart project by building the recommendation and evaluation layer that:

- Loads all Activity 9 output files and constructs learner profiles.
- Implements a Content-Based Recommender using TF-IDF similarity and difficulty-level filtering.
- Implements Collaborative Filtering using SVD Matrix Factorization and a Neighbourhood model.
- Integrates Sentiment Analysis on user reviews to produce a Hybrid Recommender.
- Evaluates all three systems using Precision@5, Recall@5, and NDCG@5, and produces a final project report.

Tasks to be Performed

Task 1: Load Activity 9 Outputs & Build Learner Profiles

1A – Load All Activity 9 Files

- Load the following files produced in Activity 9:
 - tfidf_matrix.json – TF-IDF feature vectors for all content items
 - pca_features.json – PCA-reduced features
 - vsm_index.json – VSM retrieval index
 - bm25_index.json – BM25 retrieval index
 - classified_content.json – content items with predicted difficulty labels
- Print a summary: total items loaded, difficulty distribution (Easy / Moderate / Tough counts).

1B – Simulate Learner Profiles & User-Item Interactions

- Create at least 3 learner profiles (simulate or hard-code):
 - Learner A: level = Beginner, past_interactions = [C001, C004, C008]
 - Learner B: level = Intermediate, past_interactions = [C002, C005, C007]
 - Learner C: level = Advanced, past_interactions = [C003, C006]
- Simulate a user-item rating matrix: at least 20 learners × 30 content items, ratings 1–5 (use numpy random with a fixed seed for reproducibility).
- Save the rating matrix as user_item_matrix.json.

Task 2: Content-Based Recommender System

2A – TF-IDF Similarity Recommendations

- For each learner profile, compute cosine similarity between the learner's past interaction vectors (averaged) and all content TF-IDF vectors.
- Filter recommendations by learner level:
 - Beginner → recommend only Easy content
 - Intermediate → recommend only Moderate content

- Advanced → recommend only Tough content
- Return the Top 5 recommended content items for each learner with similarity scores.

2B – Display & Save

- Display the Top 5 results for all 3 learner profiles in a formatted table: Rank | doc_id | content_title | difficulty | similarity_score.
- Save recommendations as content_based_recommendations.json.

Task 3: Collaborative Filtering & Matrix Factorization

3A – SVD Matrix Factorization

- Apply Truncated SVD (sklearn.decomposition.TruncatedSVD) on the user-item rating matrix.
- Reconstruct the predicted rating matrix.
- For a target learner (choose Learner B), identify items not yet interacted with and predict their ratings.

Return the Top 5 recommended items by predicted rating.

3B – Neighbourhood Model (User-User Similarity)

- Compute cosine similarity between all learner vectors in the user-item matrix.
- Find the Top 3 most similar learners to the target learner.
- Aggregate their highest-rated items (not yet seen by the target learner) to generate Top 5 recommendations.

3C – SVD vs Neighbourhood Comparison

- Compare Top 5 results from SVD and Neighbourhood for the same target learner.
- Discuss: Which approach is better for a new learner with few interactions? (cold-start problem)
- Save results as cf_recommendations.json.

Task 4: Sentiment Analysis & Hybrid Recommender

4A – Sentiment Analysis on User Reviews

- Simulate or collect user review text for at least 10 content items (2–3 sentences each).

- Apply Sentiment Analysis using VADER (nltk.sentiment.vader.SentimentIntensityAnalyzer) or TextBlob.
- Classify each review as Positive (compound ≥ 0.05), Neutral, or Negative (compound ≤ -0.05).
- Compute a sentiment score per content item (average compound score across all reviews).
- Save sentiment scores as sentiment_scores.json.

4B – Sentiment-Adjusted Recommendations

- Apply sentiment adjustment to the content-based recommendation list from Task 2:
 - Boost items with positive sentiment (multiply score by $1 + \text{sentiment_score}$)
 - Demote items with negative sentiment (multiply score by $1 + \text{sentiment_score}$, which reduces if negative)
- Display the adjusted Top 5 list and compare with the original Task 2 list for the same learner.

4C – Hybrid Recommender

- Build a Hybrid Recommender combining all three signal types:

Parameter	Default	Meaning
alpha (Content-Based score)	0.4	Rewards topical relevance to learner's query
beta (CF score)	0.4	Rewards items liked by similar learners
gamma (Sentiment boost)	0.2	Boosts positively reviewed items

- Formula: $\text{hybrid_score} = (\alpha \times \text{content_score}) + (\beta \times \text{cf_score}) + (\gamma \times \text{sentiment_boost})$
- Students may tune alpha, beta, gamma (must sum to 1.0) and justify their choices.
- Generate final Top 5 hybrid recommendations for each learner profile.
- Display a score breakdown table: doc_id | content_score | cf_score | sentiment_boost | hybrid_score.
- Save as hybrid_recommendations.json.

Task 5: Evaluation, Comparison & Final Project Report

5A – Evaluation Metrics

Evaluate all three recommendation systems (Content-Based, Collaborative Filtering, Hybrid) using the following metrics for K=5:

- Precision@5: fraction of top-5 recommendations that are relevant.
- Recall@5: fraction of all relevant items that appear in the top-5.
- NDCG@5 (Normalised Discounted Cumulative Gain): measures ranking quality (higher-ranked relevant items score more).
- Use the user-item rating matrix as ground truth: items rated ≥ 4 are considered 'relevant'.

5B – Comparison Table

- Create a comparison table with columns: System | Precision@5 | Recall@5 | NDCG@5.
- Rows: Content-Based | Collaborative Filtering | Hybrid Recommender.
- Identify the best-performing system and explain why.
- Discuss: advantages and drawbacks of each approach (link to Unit 5.3 topics).

5C – Final Project Report (400–600 words)

- Complete LearnSmart system architecture diagram (Activity 9 + Activity 10 combined pipeline).
- Design decisions: choice of classifiers, retrieval models, CF method, and hybrid weights.
- Results summary: sample recommendations for all 3 learner profiles with scores.
- Limitations: what the current system cannot do (e.g., cold-start, scalability, real-time updates).
- Future improvements: suggest at least 3 enhancements (e.g., deep learning embeddings, A/B testing, real user logs).

5D – Demo

- Prepare demo screenshots or a short screen recording showing:
 - A sample content-based recommendation output for one learner.

- A sample hybrid recommendation output with score breakdown.
- The evaluation comparison table.

Expected Outcome

Students should successfully:

- Build a working content-based, collaborative, and hybrid recommender system for e-learning content.
- Integrate sentiment analysis to produce sentiment-adjusted recommendations.
- Evaluate and compare all three systems using standard IR metrics (Precision@5, Recall@5, NDCG@5).
- Deliver a complete, portfolio-ready LearnSmart system spanning both Activity 9 and Activity 10.

What Students Must Submit

- Python code for all 5 tasks (single .py file or Jupyter notebook)
- Output files: user_item_matrix.json, content_based_recommendations.json, cf_recommendations.json, sentiment_scores.json, hybrid_recommendations.json
- Evaluation comparison table (screenshot or printed output)
- Final project report (400–600 words) with system architecture diagram
- Demo screenshots or short screen recording of the recommendation system in action

Evaluation Rubric

Task	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	No Attempt (0)
Task 1: Load Activity 9 Outputs & Build User Profiles	All 5 files load correctly; user profiles created with level and history; summary statistics printed.	Minor loading or profile issues.	Files loaded; basic profile created.	Partial loading; no profile.	Not attempted.
Task 2: Content-Based Recommender System	Correct TF-IDF similarity; top-5 results personalised by learner level; difficulty filtering accurate.	Minor similarity or filtering issues.	Basic content-based recommendations produced.	Incorrect similarity or no filtering.	Not attempted.
Task 3: Collaborative Filtering & Matrix Factorization	Correct SVD and neighbourhood model; top-5 recommendations generated; SVD vs neighbourhood compared clearly.	Minor matrix or comparison issues.	One CF method implemented; basic results.	Incorrect CF implementation.	Not attempted.
Task 4: Sentiment Analysis & Hybrid Recommender	Sentiment correctly classified; scores influence recommendations; hybrid formula applied; score breakdown shown.	Minor sentiment or weighting issues.	Sentiment attempted; basic hybrid built.	Sentiment incorrect; no hybrid.	Not attempted.
Task 5: Evaluation, Comparison & Final Report	Precision@5, Recall@5, NDCG@5 computed for all 3 systems; comparison table clear; report insightful (400–600 words).	Metrics mostly correct; good report.	Some metrics computed; basic report.	Few metrics; incomplete report.	Not attempted.